

Contents

The LinkedIn Launch Guide — <i>Introduction to Software Engineering in the Age of AI</i>	1
0. The facts every post must stay consistent with	1
1. How to use this guide	2
LinkedIn posting mechanics (apply to every slot)	2
Engagement rules that turn a post into contributions	2
2. The at-a-glance timetable	2
3. The posts	3
Day 0 — Launch	3
Day 1 — Lower the first-contribution friction	4
Day 2 — The single idea, on its own	4
Day 4 — “Start where I started” (build-along invite)	5
Day 6 — Three front ends, one core	5
Day 7 — The three altitudes of “correct”	6
Day 9 — What’s <i>deliberately</i> not in it	6
Day 10 — For educators: reuse and fork	7
Day 11 — Tools, introduced as you go	8
Day 13 — Two weeks in: what to fix next	8
Day 14 — Wrap + the next 90 days	9
4. Starter issues to open <i>before</i> Day 0	9
5. Measurement — what “working” looks like	10
6. Do / Don’t	10

The LinkedIn Launch Guide — *Introduction to Software Engineering in the Age of AI*

A two-week plan for getting the book read **and getting the first contributions moving**. v1.0.0 is live, so the ramp starts **Day 0 = now**. Post on relative day offsets from your launch day; the “Best time” column is a default, not a rule.

Primary goal of this campaign: feedback + contributions — open issues, first PRs, course reuses (the book is CC BY 4.0), and “here’s what I’d teach differently.” That, not vanity shares, is what we’re optimizing every post toward.

0. The facts every post must stay consistent with

Pull these from the book; don’t restate the wrong version.

- **Title:** *Introduction to Software Engineering in the Age of AI*
- **Author:** Robert Ioffe, Portland, Oregon. First edition, dated September 1, 2026.
- **License:** CC BY 4.0 — free, open, and reusable; anyone may teach from it or fork it, with attribution.
- **Links** (use the short, stable forms):
 - Read online: https://rioffe.github.io/intro_to_ai_software_engineering/
 - PDF / HTML: https://github.com/rioffe/intro_to_ai_software_engineering/releases/tag/v1.0.0
 - Source + issues + PRs: https://github.com/rioffe/intro_to_ai_software_engineering
- **Shape:** 14 chapters. One system — a “hybrid” mortgage calculator — built up from an empty repo to three front ends (CLI, GUI, a model that can operate it). Tools (terminal, git, Python, a coding agent, local + hosted models) introduced right when a chapter needs them.
- **The through-line: a human-verified core, an AI-assisted layer.** You design/verify the core; an agent drafts and a model reasons. “Correct” stays yours, written three times at three altitudes — a plain-language **spec**, executable **tests**, a formal **schema** a model can call.

- **Reality check for CTAs:** the repo today has **no ISSUE_TEMPLATE** and **no PULL_REQUEST_TEMPLATE**. So an early CTA that says “here’s the format your first issue/PR can fill in” is a genuine, welcome task — not a trap. Lean into that.

1. How to use this guide

1. Post the **Day 0** copy first. Then keep one post every 1–2 days through Day 14.
2. Each slot gives you **purpose, audience angle, timing, the full post copy, and the follow-through** (what you do in the comments within the first hour — this is where LinkedIn’s early algorithm reward actually lives).
3. Every post ends on a **concrete CTA** that routes a reader to a specific action (open an issue, file a “this step broke” report, reuse for a course, propose a change). Don’t post a slot with its CTA missing — the CTA is the whole point.
4. Copy is shown in “text blocks so hashtags/links stay intact. ****Paste the text, not the fence.**** Strip the code-fence lines and any leading<...>’ before pasting into LinkedIn.

LinkedIn posting mechanics (apply to every slot)

- **Best window:** weekday 7:30–9:00 AM *or* 12:00–1:30 PM, reader’s local time. Evenings (7–9 PM) on Tue/Thu are a respectable second window. **Not** weekends, not Fridays after lunch.
- **First 60 minutes matter more than the total run.** Reply to every comment within the first hour; post a comment of your own at ~30 min (“here’s what I’d start with if you’re coming in cold to link to Chapter 0).
- **Hashtags: 3–5, at the end, no leading @.** More than 5 reads as spam.
- **No link-only posts.** Put the value in the post; the link is the finish, not the payload.
- **One idea per post.** A post that also asks “hey, buy my next book / follow me / rate” is a post that asks nothing and gets nothing.
- **Tag people only when you’ve actually engaged them.** Pinging “big names you have never spoken to” is a flag.
- **The reply-to-your-own-post trick, used well:** pin a genuinely useful comment (a 2-minute “do this first” path, or a link to the most relevant issue) — LinkedIn surfaces it to new readers. Don’t spam self-replies.

Engagement rules that turn a post into contributions

- **Seed the contribution surfaces yourself, first.** On Day 0/1, *you* open 2–3 starter issues (below) so the repo isn’t empty when a CTA sends people there. An empty Issues tab is the single fastest way to lose a would-be contributor.
- **Turn every good comment into a public issue + credit.** “You caught a gap in Chapter 7 — opened issue #12, thanks for the eyes.” Public, named, fast. This is the behavior you’re modeling.
- **The “here’s a task sized to take 20 minutes” post outperforms “come review the book.”** Name the exact 20-minute task.

2. The at-a-glance timetable

Day	Post	Core CTA
0	Launch announcement	“Clone it, run the first build, tell me what breaks”
1	“Here’s the format your first issue/PR can set”	Open a starter issue or PR template
2	The one idea the book is built on	“Disagree? Open an issue and argue it.”

Day	Post	Core CTA
4	“Start where I started” (build-along)	“Hit Chapter 0’s checkpoint, report where it stuck”
6	Three front ends, one core	“Propose a 4th front end as an issue”
7	The three altitudes of “correct”	“Which of the three is yours hardest to keep straight?”
9	What’s deliberately <i>not</i> in it	“Name one out-of-scope tool I should document; we PR the appendix”
10	For educators: reuse & fork it	“Adapt it for a course; open a reuse issue”
11	Tools, introduced as you go	“What would you introduce <i>differently?</i> ” (issue)
13	Two weeks in — what to fix next	“Vote on the open issues / open a new one”
14	Wrap + the next 90 days	“Star it / follow the issues / take the first maintainer hat”

(Skip a day if a slot has no live material to back its CTA. A cadence you can’t sustain for 14 days at lower intensity beats a sprint that dies at Day 6.)

3. The posts

Day 0 — Launch

- **Purpose:** existence + “come use it and tell me what’s wrong.”
- **Angle:** between the two camps of “AI coding” material; here’s the in-between.
- **Time:** Tue/Thu, 8:00 AM.

I wrote a book, and it's just gone live: *Introduction to Software Engineering in the Age of AI*.

Free, online, CC BY 4.0. I'd rather your first read also be your first fix.

Most "AI coding" material lands in one of two camps: prompt tricks that stop working next month, or hand-wringing about whether any of this is engineering at all. I tried the in-between - software engineering taught as a craft, with AI as a tool you learn to hold correctly.

So it's built the way you'd actually build: one system, one chapter per step. You start at an empty repository and end with a hybrid mortgage calculator that has three front ends - a command line, a graphical UI, and a language model that can run the whole thing on your behalf.

The through-line is one idea, repeated: a human-verified core, an AI-assisted layer. You design, write, and verify the core yourself; an agent drafts and a model reasons around it - but neither one gets to decide what "correct" means. That judgment stays with you, written three times at three altitudes: a plain-language spec, executable tests, and a formal schema a model can call.

Here's the ask. The book ships 14 chapters of "do this, then verify this" - and a first-edition book always has steps I wrote but never had run on your machine. So: clone it, do Chapter 0, and if a beat doesn't work for you, open an issue. That's the most useful thing you can do with it today.

- Read online: https://rioffe.github.io/intro_to_ai_software_engineering/
- PDF and HTML: https://github.com/rioffe/intro_to_ai_software_engineering/releases/tag/v1.0.0
- Source, issues, and pull requests: https://github.com/rioffe/intro_to_ai_software_engineering

If you teach from it, it's yours to teach with (CC BY 4.0). Come build it with me.

#SoftwareEngineering #AI #OpenEducation #Programming

Follow-through (first hour):

- Pre-open 3 starter issues (see §4) so the Issues tab isn't empty when people land.
- Post your own pinned reply: "If you're here and unsure where to start — Chapter 0.7 sets up the project with uv; if that beat trips you up, that's a perfectly good thing to file. Here's the link: ..."
- Reply to everyone, within ~60 min.

Day 1 — Lower the first-contribution friction

- **Purpose:** make the very first issue/PR *easy and legitimate* (there are no templates yet).
- **Angle:** "the repo has no issue template — so your first issue can become the template."
- **Time:** 8:00 AM.

Small thing, but it's the thing that kills most first-timers.

My book's GitHub repo has no issue template and no PR template. Which sounds like a chore, but it means the first issue you file gets to also be the format everyone files in after you.

Three good issues to open today, one per box:

- "Chapter 0.7 - a step didn't run for me" (put your OS + exact output in it).
- "This chapter jumps too fast / too slow at [page]."
- "The PDF says X, the online build says Y" (a real cross-edition mismatch to check).

Any of these is a real first PR. If you're unsure what counts, that uncertainty itself is the issue worth opening.

- Source, issues, and pull requests: https://github.com/rioffe/intro_to_ai_software_engineering

#OpenSource #Contribute #SoftwareEngineering

Follow-through:

- For each issue a reader opens, reply same day, and — if it's the *format* of it — link it as the de-facto template in a new comment. This public "you started this" credit is the behavior the campaign is trying to teach.

Day 2 — The single idea, on its own

- **Purpose:** give people a sentence to react to/argue with. Ideas travel on disagreement.
- **Angle:** "I'm going to be specific; argue with me in the issues."
- **Time:** Wed 12:30 PM.

The whole book rests on one sentence. I'm putting it alone so you can disagree with it.

A book that teaches you to let an agent write everything teaches you to trust something you can't yet evaluate. A book that bans AI teaches a world where the tools in Chapter 1

don't exist. Both are wrong.

So the book teaches the boundary instead: a human-verified core, an AI-assisted layer. You own the core; the agent drafts; the model reasons. And "correct" is not delegated - it's written down three times: a plain-language spec, executable tests, and a formal schema a model can call. Same contract, three altitudes.

Where does that boundary actually sit, in *your* day-to-day work? That's the question the book keeps asking itself - and it'd be a good issue to open, arguing one way or the other.

- Issues tab: https://github.com/rioffe/intro_to_ai_software_engineering/issues

#SoftwareEngineering #AI #Programming

Follow-through:

- Reply to every "I'd move that boundary [here]" comment by naming the exact chapter that touches it. This is the campaign's most comment-dense slot.
-

Day 4 — "Start where I started" (build-along invite)

- **Purpose:** move readers past the reading list and into actually running the thing.
- **Angle:** "do just the next 30 minutes, then tell me."
- **Time:** Thu 8:00 AM.

You don't need to read the whole book before it's useful. Try this instead.

Open Chapter 0, do 0.7 (the project setup with `uv`). When it's done, run the first checkpoint the book hands you - a line or two that says what your project should be like right now. Then compare. Either you match it, or you don't.

If you match it: great, file a 1-line "clean run on [OS]" as an issue, it helps the next reader.

If you don't: that's the most useful thing you can do with the book. Open an issue with your OS, your version, and the exact error. That's a contribution, not a failure.

- Read online: https://rioffe.github.io/intro_to_ai_software_engineering/

#SoftwareEngineering #Python #OpenEducation

Follow-through:

- For each "clean run on [OS]" issue, link it from the book's "reported on" list when it gets merged — that closes the loop and makes the *behavior* visible, not just the outcome.
-

Day 6 — Three front ends, one core

- **Purpose:** the most visually shareable idea in the book. Show the architecture, not the prose.
- **Angle:** "one core, three faces — one of which is a model."
- **Time:** Tue 12:30 PM.

A small architecture decision I think about a lot: same core, different front ends.

In the book, the "core" is a pure function that takes a mortgage input and returns a validated payment. It has no idea who's calling it. On top of it sits a CLI, a PyQt5 GUI,

and - this is the interesting one - a tool interface a language model can call to operate the same calculator on your behalf.

The core stays pure. Each front end is a thin adapter around it. The model-facing adapter is just another adapter - a little stricter about the schema it exposes, because that's where a model's uncertainty lands.

If you had a 4th front end for this - a phone widget, a notebook, an email reply - how would you draw that line? Open an idea issue; I collect them here:

- Issues tab: https://github.com/riooffe/intro_to_ai_software_engineering/issues

#SoftwareEngineering #Architecture #AI

Follow-through:

- Pin a one-line “how the book actually draws it” comment (link to Chapter 10) so the thread ends grounded, not in the air.
-

Day 7 — The three altitudes of “correct”

- **Purpose:** the deepest idea in the book, made concrete.
- **Angle:** “You’ve probably written two of these already. Which is the third?”
- **Time:** Wed 12:30 PM.

Most of us keep “what's correct?” in one of three places. The book asks you to keep it in all three, at the same time.

1. A plain-language spec - one a human with a coffee can follow.
2. Executable tests - one a machine can re-run on every change.
3. A formal schema - one a language model can call and be held to.

Same contract, three altitudes. Miss one and you find out the other two are lying to you. Spec-without-tests is a wish. Tests-without-schema is a model with a private definition of correct. Schema-without-spec is a form that no one, human or machine, can argue about.

Which of these three do *you* keep the least? That one is usually the one your system already has a hole in. Open an issue naming it - it'll also be good fuel for Chapter 2 revisions.

#SoftwareEngineering #Testing #AI

Follow-through:

- For each “I keep least [X]” reply, name a chapter where that altitude first shows up — turns the thread into a navigation aid for the book.
-

Day 9 — What’s *deliberately* not in it

- **Purpose:** the out-of-scope list in the book’s Appendix is a great PR queue for readers.
- **Angle:** “I left four useful tools out on purpose; you can write the notes I didn’t.”
- **Time:** Thu 8:00 AM.

One thing I want to be honest about: this book intentionally skips a few tools - Docker, CI/CD, mypy, pre-commit. Not because they don't matter, but because at *this* level of the

project they're overhead that gets in the way of the one thing the book is trying to teach.

Each one gets a short section in the Appendix: what it solves, why it stayed out, and where it would slot in if you decided to add it. But those sections are stubs - the real notes a reader on their *next* project would need are yours to write.

Pick one:

- Add a minimal CI/CD workflow for the project as a PR (GitHub Actions, Ruff to pytest).
- Add a small "if you want to add mypy to core.py" note as an issue.
- Add a "packaging the GUI with PyInstaller" writeup as a new Appendix section.

Any of these is a clean, self-contained first PR. The book ships in 14 chapters; the Appendix is where the readers take over.

- Source, issues, and pull requests: https://github.com/rioffe/intro_to_ai_software_engineering

#OpenSource #SoftwareEngineering #Contribute

Follow-through:

- For each Appendix PR that lands, link it from that section so the section visibly grows — that's the payoff readers see when they return.

Day 10 — For educators: reuse and fork

- **Purpose:** unlock the CC BY 4.0 angle; this is where real “contribution” volume lives.
- **Angle:** “teach it this term; here’s how to start without asking permission.”
- **Time:** Tue 8:00 AM.

If you teach this subject in the next term, you don't need to ask me first. The book and its build tooling are CC BY 4.0 - free to adapt, as long as you're attributed.

Three ways I'd expect it to get reused:

- As an assignment sequence - one chapter per week, the "definition of done" checklists are the weekly deliverables.
- In a fork - swap the mortgage calculator for a domain your course runs on, keep the architecture, drop the domain-specific bits.
- As a reference - point students at the Appendix for the "next things" (Docker, CI/CD, mypy) without you having to teach those yourself this term.

If you do any of these, open a "reuse" issue: the course, the term, the domain you swapped in, the part that landed or didn't. I'll keep a "taught in" list, and it helps the next educator find you.

- Issues tab: https://github.com/rioffe/intro_to_ai_software_engineering/issues

- Read online: https://rioffe.github.io/intro_to_ai_software_engineering/

#OpenEducation #Teaching #AI

Follow-through:

- For each “taught in” issue, reply with the chapter sequence you’d suggest for that course shape — you become the on-ramp for the next educator, which is the whole point.

Day 11 — Tools, introduced as you go

- **Purpose:** the “we introduce each tool the chapter before it’s needed” decision, and what readers would do differently with it.
- **Angle:** “One tool per beat. Which of my beats would you reorder?”
- **Time:** Wed 12:30 PM.

One decision I'm proud of: the tools in the book appear the chapter before they're used, not in a reference dump up front. The terminal in Chapter 0, git in Chapter 1, Python and `uv` in Chapter 0.7, a coding agent in Chapter 1, the first language model in Chapter 11. Every time something shows up, it's because the next chapter needs it.

This is a teaching decision, not a packaging one - I've seen books that spend 200 pages on tools and then never come back. That's not a book, that's a manual wearing a book's clothes.

So the question: where does your own mental list of "tools a new engineer needs" start? Where does yours diverge from mine? Open a "what I'd introduce before / after" issue; it's the cleanest signal for the next edition.

- Source, issues, and pull requests: https://github.com/rioffe/intro_to_ai_software_engineering

#SoftwareEngineering #Learning #Python

Follow-through:

- For each “I’d move [tool] earlier/later” issue, link to the chapter where it currently appears, and note what would have to change to move it. This is Chapter 2 material in public, and the book visibly benefits.

Day 13 — Two weeks in: what to fix next

- **Purpose:** aggregate the campaign into “here’s what 14 days of your comments told me,” and hand the next round back to readers.
- **Angle:** “You’ve caught X things in a fortnight; what’s next?”
- **Time:** Thu 8:00 AM.

Two weeks in. Here's what your issues + comments told me:

- N issues open, N merged, N in review
- The top three "step didn't run for me" reports were on [beats]
- The most requested change is [one thing]

I'm going to prioritize the next round of revisions on that list, not on my own feelings about the book. Open an issue if there's a beat you want on the list. Star the repo if it's your next read.

- Source, issues, and pull requests: https://github.com/rioffe/intro_to_ai_software_engineering

#SoftwareEngineering #SoftwareDevelopment

Note: this slot needs real numbers from the repo’s own Issues tab. Don’t post it with placeholder values — a post that says “we’ve been great” with no numbers is the one kind of content that loses the readers you just won.

Follow-through:

- After you post, open the issue that aggregates the top three “didn’t-for-me” reports — that issue *is* the next round of revisions. Link it in a reply within 30 minutes so the thread has somewhere to land.
-

Day 14 — Wrap + the next 90 days

- **Purpose:** close the loop honestly, hand the book to the next reader, and start the next cadence without pretending the work is done.
- **Angle:** “14 days, here’s what happened, here’s what’s next.”
- **Time:** Tue 8:00 AM.

The book's two weeks out in the world today. N issues, N PRs, N "taught in" reuses - all open in the Issues tab, most of them yours.

I don't think the book is finished. I think it's the point where I stop writing and start reacting - at which point the "next 90 days" looks like:

- Merging the top of the issue queue into a 1.1.
- A new chapter on [one of the most-requested Appendix topics] - open an issue if you want to write it.
- A "taught in" page that links every course that reused the book, so the next educator finds a peer who's done it.

If you got to one beat and stopped, that's fine - every reader should. Open an issue with where you got to and what stopped you, it's the best map I can get for whoever's next.

- Source, issues, and pull requests: https://github.com/rioffe/intro_to_ai_software_engineering
- Read online: https://rioffe.github.io/intro_to_ai_software_engineering/

#SoftwareEngineering #AI #OpenEducation

Follow-through:

- Pin a “how to contribute, in 60 seconds” comment that links the issue queue and the “reuse” label — this is the pin a returning reader reads three months later.
-

4. Starter issues to open *before* Day 0

An empty Issues tab is the single fastest way to lose a would-be contributor, so open these yourself **before Day 0** so the CTA has somewhere to land. They double-guess the three boxes the Day 1 post invites readers to fill:

1. [docs] **first-timer: a Chapter 0.7 step that doesn't run on [OS name]** — template for the “step didn’t run” bucket. A real, open, in-progress example beats a fake issue.
2. [content] **Chapter N to Chapter N+1: pacing gap at [specific page]** — the “jumps too fast / too slow” bucket. Pick a beat you actually know is rough.
3. [docs] **Appendix: a "next thing" note worth writing - pick one (Docker/CI-CD/mypy)** — the “you can PR the Appendix” bucket, already scoped.

Each of these three is genuinely useful on its own, doesn’t depend on a reader showing up, and models exactly the behavior the campaign wants to teach.

5. Measurement — what “working” looks like

LinkedIn’s “views / impressions / reactions” numbers are mostly noise for this goal. Track the ones that actually matter, weekly:

- **Open issues / open PRs** — this is the number that matters. A dozen open PRs > 10k views.
- **Merged PRs in the first two weeks** — proof the CTA closed the loop, not just opened one.
- **“Taught in” issues** — the long-tail; the only kind of reuse that compounds.
- **Reactions on specific posts** — only worth reading if an individual post’s reaction rate is off the mean by a lot; that post tells you something about the audience shape.
- **Direct messages / “I’ll read this” comments** — this is where the “I’ll do it later” crowd lands; follow up privately if the DM is a real question, not a compliment.

Don’t chase views. Chase the issues tab.

6. Do / Don’t

Do: - Post one idea per post, CTA at the end, 3–5 hashtags. - Reply to every comment within the first hour. - Turn public comments into public issues with credit — that’s the teaching behavior. - Post a “here’s the 60-second path” comment of your own within 30 minutes of each post. - Skip a scheduled slot rather than post an empty one.

Don’t: - Don’t post link-only posts. The link is the finish, not the payload. - Don’t tag people you’ve never engaged. - Don’t post “I just finished my book” after the launch window — it reads as noise to a LinkedIn feed full of people finishing books. - Don’t post Day 13 or Day 14 with placeholder numbers. If the numbers aren’t real yet, push the slot by a day. - Don’t mix audiences: a post aimed at educators reads flat to working engineers and vice versa. Pick the audience per post, not per campaign.

Author of record: Robert Ioffe. Posts are first-person; signature not required, but the book’s front matter reads “— Robert Ioffe, Portland, Oregon, September 1, 2026.”